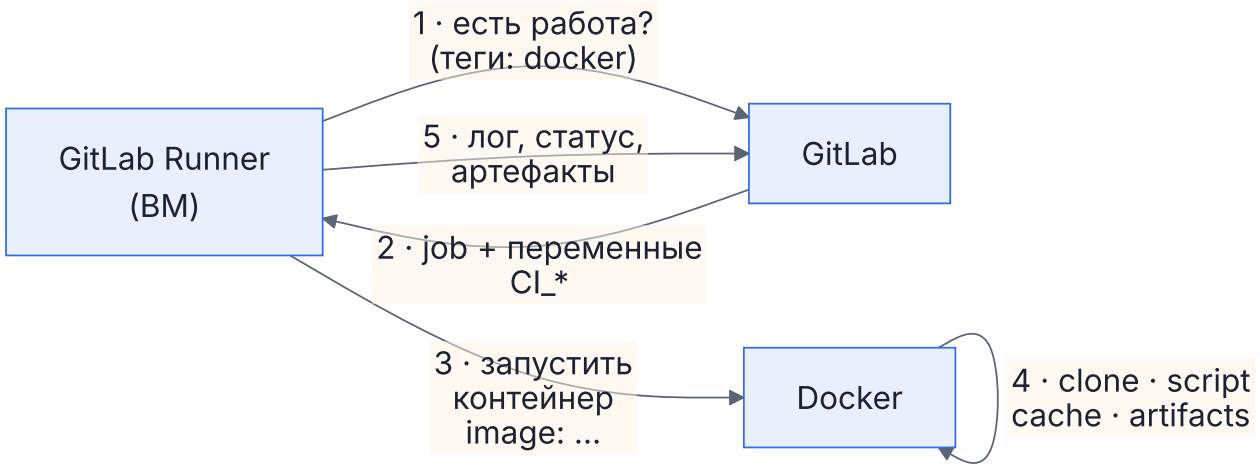
GitLab Runner и его настройка

Занятие 09 · День 4, после обеда (часть 2)

Ставим runner на отдельную виртуальную машину

Что такое GitLab Runner

- Отдельное приложение (Go), которое **выполняет jobs** пайплайнов
- Сам **опрашивает** GitLab: «есть работа для меня?» — входящие порты на runner не нужны
- Может работать где угодно: ВМ, железо, Docker, Kubernetes, ноутбук
- Один runner-процесс обслуживает несколько **runner'ов** (записей в `config.toml`)



Типы runner'ов по области

Тип	Кто создаёт	Кому доступен
Instance	администратор	всем проектам GitLab
Group	Owner группы	всем проектам группы и подгрупп
Project	Maintainer проекта	одному проекту

Плюс настройки:

- **Tags** — job с `tags: [docker]` возьмёт только runner с тегом `docker`
- **Run untagged jobs** — брать ли job без тегов
- **Protected** — только пайплайны защищённых веток/тегов (для деплой-runner'ов!)
- **Paused** — временно не брать работу

Executor — как runner выполняет job

Executor	Как	Когда
`shell`	прямо в ОС runner'а	просто, но «грязно»: зависимости копятся
`docker`	каждый job в чистом контейнере `image:`	**самый популярный**
`docker-autoscaler` / `instance`	создаёт ВМ в облаке под задачи	автомасштабирование
`kubernetes`	каждый job — под в кластере	k8s-платформы
`ssh`, `virtualbox`, `parallels`, `custom`	особые случаи	редко

`docker+machine` — устарел, заменён на `docker-autoscaler`.

Почему отдельная ВМ

- **Изоляция**: сборки не мешают GitLab и кластеру (CPU, диск, сеть)
- **Безопасность**: job выполняет **чужой код** из репозитория
- **Сборка образов**: нужен Docker-демон — в кластере это privileged-контейнеры
- Масштабирование: добавил ВМ — добавил runner

```
# на хосте
vagrant up gitlab-runner
vagrant ssh gitlab-runner
```

`gitlab-runner` — 192.168.56.31, Ubuntu 24.04, 2 CPU, 1.5 GB

ДЕМО Шаг 1. Docker Engine

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
```

Registry GitLab на стенде — по **HTTP**, разрешаем его явно:

```
sudo tee /etc/docker/daemon.json <<'EOF'
{ "insecure-registries": ["registry.192.168.56.200.nip.io"] }
EOF
sudo systemctl restart docker
docker info | grep -A2 "Insecure Registries"
```

ДЕМО Шаг 2. GitLab Runner

```
curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh" \
  -o script.deb.sh
less script.deb.sh              # всегда смотрим, что запускаем от root
sudo bash script.deb.sh
sudo apt-get install -y gitlab-runner

gitlab-runner --version
systemctl status gitlab-runner --no-pager
sudo usermod -aG docker gitlab-runner
```

Сервис `gitlab-runner` работает от root, конфигурация — `/etc/gitlab-runner/config.toml`.

Регистрация: новый процесс

Было (до GitLab 16): общий *registration token* проекта → `gitlab-runner register --registration-token …`
→ устарело, **удаляется в GitLab 20.0**

Стало: runner создаётся **в UI** (или через API) → получаем **authentication token** `glrt-…`

1. Project → **Settings → CI/CD → Runners → New project runner**
2. Tags: `docker` · Run untagged jobs: нет · Description: `vm-docker-runner`
3. **Create runner** → копируем команду с токеном `glrt-…`

> Токен показывается **один раз**. Хранится в `config.toml` — файл с правами 600.

ДЕМО Шаг 3. Регистрация

```
sudo gitlab-runner register \
  --non-interactive \
  --url "http://gitlab.192.168.56.200.nip.io" \
  --token "glrt-XXXXXXXXXXXXXXXX" \
  --executor "docker" \
  --docker-image "alpine:3.22" \
  --docker-volumes "/var/run/docker.sock:/var/run/docker.sock" \
  --docker-volumes "/cache" \
  --description "vm-docker-runner"

sudo gitlab-runner verify
sudo gitlab-runner list
```

В GitLab: Settings → CI/CD → Runners — зелёный кружок **online**.

config.toml

```
concurrent = 2           # сколько jobs одновременно на этой машине (всего)
check_interval = 3       # как часто спрашивать GitLab, сек

[[runners]]
  name = "vm-docker-runner"
  url = "http://gitlab.192.168.56.200.nip.io"
  token = "glrt-XXXXXXXX"
  executor = "docker"
  [runners.cache]
    MaxUploadedArchiveSize = 0
  [runners.docker]
    image = "alpine:3.22"          # если в job не указан image
    privileged = false
    pull_policy = ["if-not-present"]
    volumes = ["/var/run/docker.sock:/var/run/docker.sock", "/cache"]
    shm_size = 0
```

Изменения подхватываются **без перезапуска** (runner перечитывает файл).

Как собирать Docker-образы в CI

Способ	Как	Плюсы / минусы
Проброс сокета `docker.sock`	контейнер job управляет Docker хоста	просто, быстро, кэш слоёв; **job = root на хосте**
Docker-in-Docker `services: [docker:dind]`	отдельный демон в privileged-контейнере	изоляция между jobs; privileged, нет кэша
Buildah / BuildKit rootless	сборка без демона	безопаснее, подходит для Kubernetes

На стенде — **docker.sock** (отдельная ВМ, только наш код).
В общих runner'ах — dind или rootless-сборщики.

> Runner с `docker.sock` или `privileged = true` никогда не делайте instance-runner'ом для чужих проектов.

ДЕМО Первый пайплайн

```yaml
stages: [test]

default:
  tags: [docker]

hello:
  stage: test
  image: alpine:3.22
  script:
    - echo "Привет из GitLab CI!"
    - cat /etc/os-release | head -2
    - echo "Коммит $CI_COMMIT_SHORT_SHA в ветке $CI_COMMIT_BRANCH"
    - hostname
```

Commit → **Build → Pipelines** → job → лог. Смотрим, как runner скачивает образ, клонирует репозиторий и выполняет `script`.

Что делает runner при каждом job

1. Получает job: скрипт, переменные `CI_*`, образ, кэш/артефакты
2. Скачивает `image` (с учётом `pull_policy`)
3. Запускает служебный **helper**-контейнер: `git clone` по `CI_JOB_TOKEN`, кэш, артефакты
4. Выполняет `before_script` + `script` + `after_script` в контейнере `image`
5. Стримит лог в GitLab, загружает артефакты, удаляет контейнеры

```
sudo journalctl -u gitlab-runner -f      # на BM runner'а
docker ps -a                              # контейнеры runner-*
```

Runner в Kubernetes — для сравнения

```
helm repo add gitlab https://charts.gitlab.io
helm install gitlab-runner gitlab/gitlab-runner -n gitlab-runner --create-namespace \
  --set gitlabUrl=http://gitlab.192.168.56.200.nip.io \
  --set runnerToken=glrt-XXXX \
  --set rbac.create=true
```

- Executor **kubernetes**: каждый job — отдельный под в кластере
- Масштабируется вместе с кластером, ресурсы jobs — через requests/limits
- Сборка образов — rootless-сборщики (BuildKit, Buildah)

На стенде не ставим — бережём память; принцип тот же.

Обслуживание и безопасность

- `concurrent` — по числу CPU, не больше
- Чистка: `docker system prune` по cron или `clear-docker-cache` (пакет runner'а)
- Обновлять runner вместе с GitLab (мажорные версии должны совпадать)
- **Protected runner** для деплоя в production
- Секреты — в **CI/CD-переменных** (masked, protected), не в `.gitlab-ci.yml`
- Не давать runner'у с доступом к production брать jobs из MR от форков

Если job не стартует

Симптом	Причина
`pending`, «no runners online»	runner offline / не тот проект / не совпали теги
`pending` у job без тегов	у runner'а выключено «Run untagged jobs»
`Cannot connect to the Docker daemon`	нет `docker.sock` в volumes / пользователь не в группе docker
`http: server gave HTTP response to HTTPS client`	registry не в `insecure-registries`
`could not resolve host gitlab…nip.io`	DNS на BM runner'а
`This job is stuck … protected`	job из незащищённой ветки, runner protected

Итоги

- **Runner** опрашивает GitLab и выполняет jobs; GitLab сам ничего не запускает
- Область: instance / group / **project**; выбор по **тегам**
- Executor **docker** — чистая среда для каждого job
- Регистрация — через UI + токен `glrt-` (registration tokens уходят в 20.0)
- `config.toml`: `concurrent`, `[runners.docker]`, volumes
- Сборка образов: docker.sock / dind / rootless — осознанный выбор

Практика 09

Runner для своего проекта

Задание — на платформе testlms.kz